
Software Requirements Specification

for

**VoiceHire – Connecting Customers and
Skilled Workers Through Voice**

Version 1.0 approved

Prepared by Subharanjani.D

Department of Computer Science and Engineering,

Coimbatore Institute of Technology

15th October 2026

Table of Contents

Table of Contents.....	ii
Revision History.....	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope.....	1
1.5 References.....	2
2. Overall Description.....	2
2.1 Product Perspective.....	2
2.2 Product Functions.....	3
2.3 User Classes and Characteristics.....	3
2.4 Operating Environment.....	8
2.5 Design and Implementation Constraints.....	8
2.6 User Documentation.....	8
2.7 Assumptions and Dependencies.....	8
3. External Interface Requirements.....	9
3.1 User Interfaces.....	9
3.2 Hardware Interfaces.....	9
3.3 Software Interfaces.....	9
3.4 Communications Interfaces.....	10
4. System Features.....	10
4.1 User Registration and Login.....	10
4.2 Worker Profile Management.....	11
4.3 Voice-Based Service Request.....	11
4.4 Worker Search and Matching.....	12
4.5 Job Request Management.....	12
5. Other Nonfunctional Requirements.....	13
5.1 Performance Requirements.....	13
5.2 Safety Requirements.....	13
5.3 Security Requirements.....	13
5.4 Software Quality Attributes.....	13
5.5 Business Rules.....	13
6. Other Requirements.....	14
Appendix : Analysis Models.....	14
E-Resource.....	43

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for **VoiceHire – Connecting Customers and Skilled Workers Through Voice, Version 1.0**. The system enables customers to find and hire skilled workers such as plumbers, electricians, carpenters, painters, and technicians through voice-based interaction. This SRS covers all functional and non-functional requirements, user interfaces, system features, and constraints of the application.

1.2 Document Conventions

- Section headings are numbered hierarchically.
- Functional requirements are identified using FR-x.
- Non-functional requirements are identified using NFR-x.
- Important terms are highlighted in **bold**.
- Requirements are written using the keyword "shall" to indicate mandatory functionality.
- Priority levels are classified as High, Medium, or Low.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Project Developers
- System Designers
- Testers and QA Engineers
- Project Managers
- End Users
- Documentation Writers

Readers should begin with the Introduction and Overall Description sections before proceeding to detailed requirements and system specifications.

1.4 Product Scope

VoiceHire is a multilingual voice-assisted platform that connects customers with skilled workers. Users can search for services using voice commands in their preferred language, while workers can register and receive job opportunities. The system aims to simplify job discovery, reduce communication barriers, and improve employment opportunities for blue-collar workers.

Objectives:

- Enable voice-based job searching.
- Support multiple regional languages.
- Connect customers and workers efficiently.
- Improve accessibility for users with limited literacy.

1.5 References

- IEEE Recommended Practice for Software Requirements Specifications (IEEE 830).
- Android Developer Documentation.
- Spring Boot Documentation.
- MySQL Database Documentation.
- Google Speech-to-Text API Documentation.
- Project Vision and Scope Document, Version 1.0.

2. Overall Description

2.1 Product Perspective

VoiceHire is a standalone web and mobile application designed to bridge the gap between customers and skilled workers. Existing platforms often require users to read and type. VoiceHire introduces voice-based interactions, making the system accessible to users with varying literacy levels.

System Components:

- Customer Interface
- Worker Interface
- Admin Dashboard
- Voice Recognition Module
- Database Server
- Notification Service

2.2 Product Functions

The system shall provide:

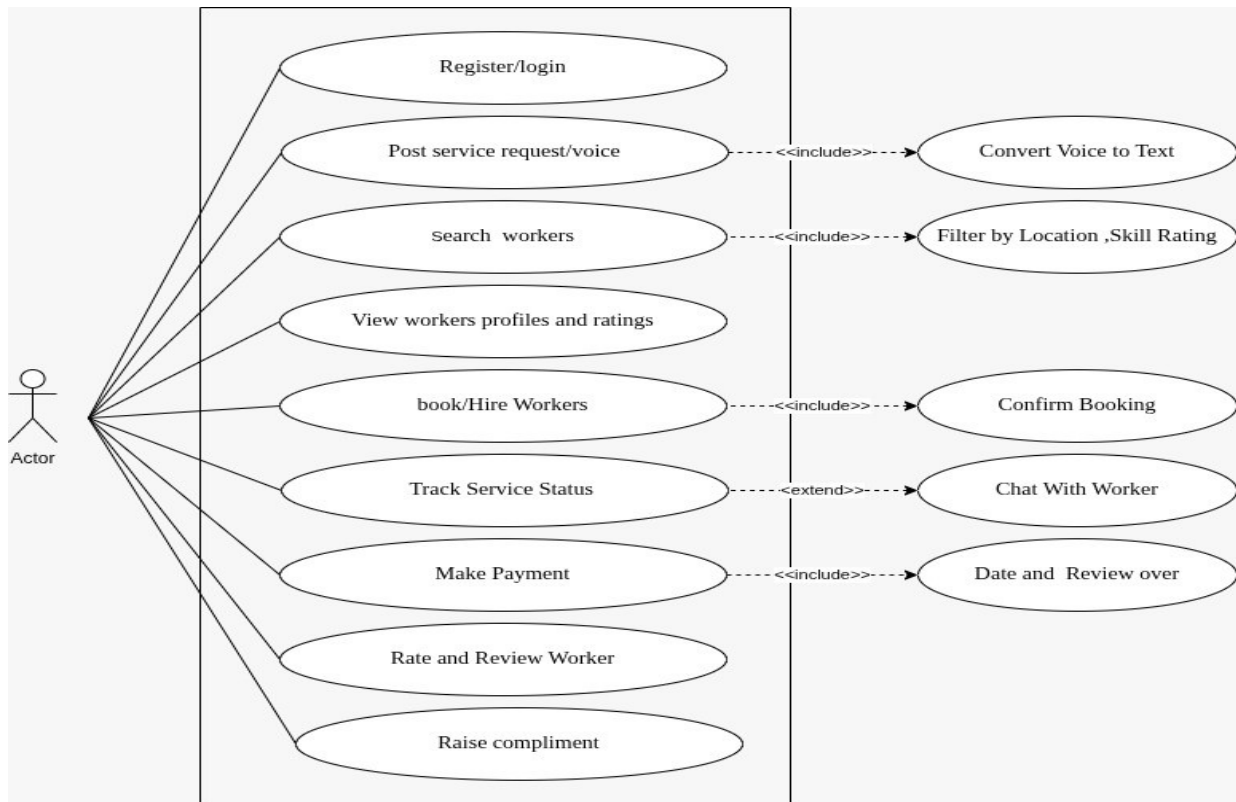
- User Registration and Login
- Worker Profile Creation
- Voice-Based Service Search
- Multilingual Support
- Job Posting by Customers
- Job Acceptance by Workers
- Real-Time Notifications
- Ratings and Reviews
- Location-Based Worker Search
- Admin Monitoring and Management

2.3 User Classes and Characteristics

2.3.1.1 Users and usecase diagram

- Service provide(Client)
- Job seekers (Blue Collar People)
- Admin(platform manager).
- Guest User

Service provide(Client) :



Voice Processing Module :

Module Name: Voice Processing

Primary Actor: Customer

Secondary Actor: System

Preconditions: Microphone permission granted

Postconditions: Voice converted into text

Trigger: User presses voice button

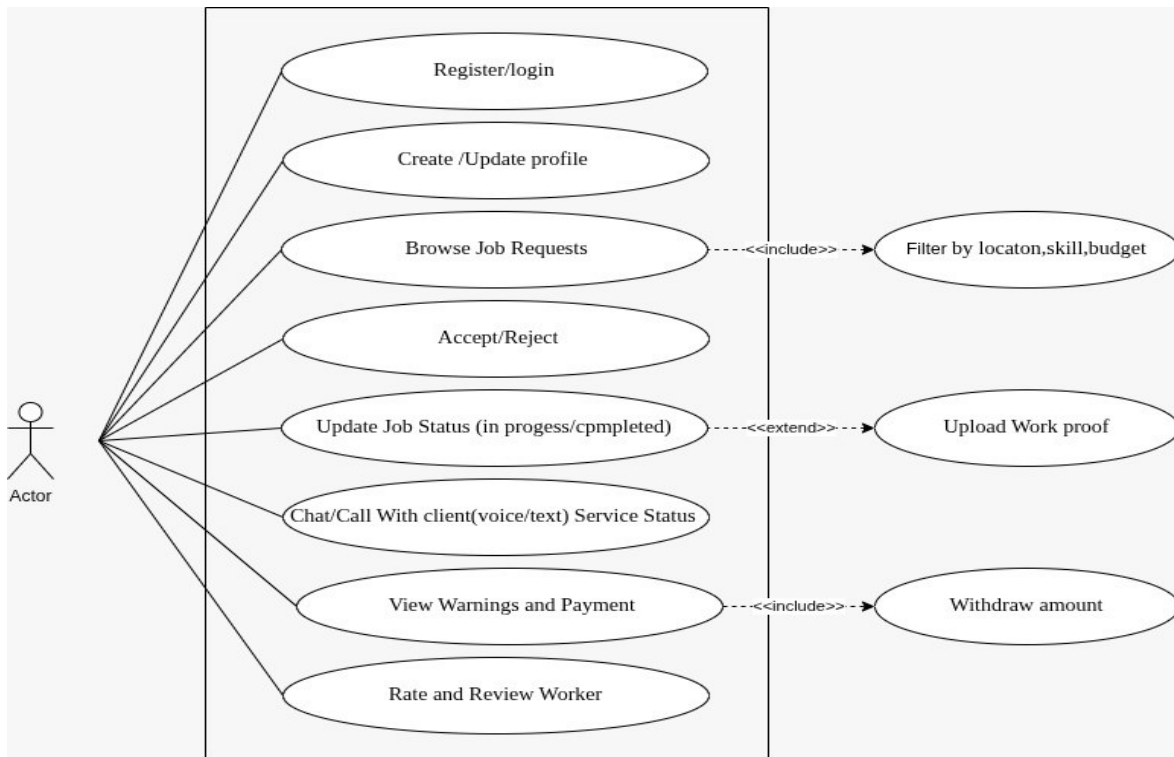
Main Success Scenario:

User clicks microphone → system captures voice → converts to text → processes meaning → displays correct output → sends data to next module

Exception Scenario:

Microphone denied OR unclear speech → system fails to process → error shown → user retries or uses text input

Job seekers (Blue Collar People)



Job matching Module:

Module Name: Job Matching

Primary Actor: System

Secondary Actor: Customer / Worker

Preconditions: Job exists

Postconditions: Matching workers displayed

Trigger: Job posted

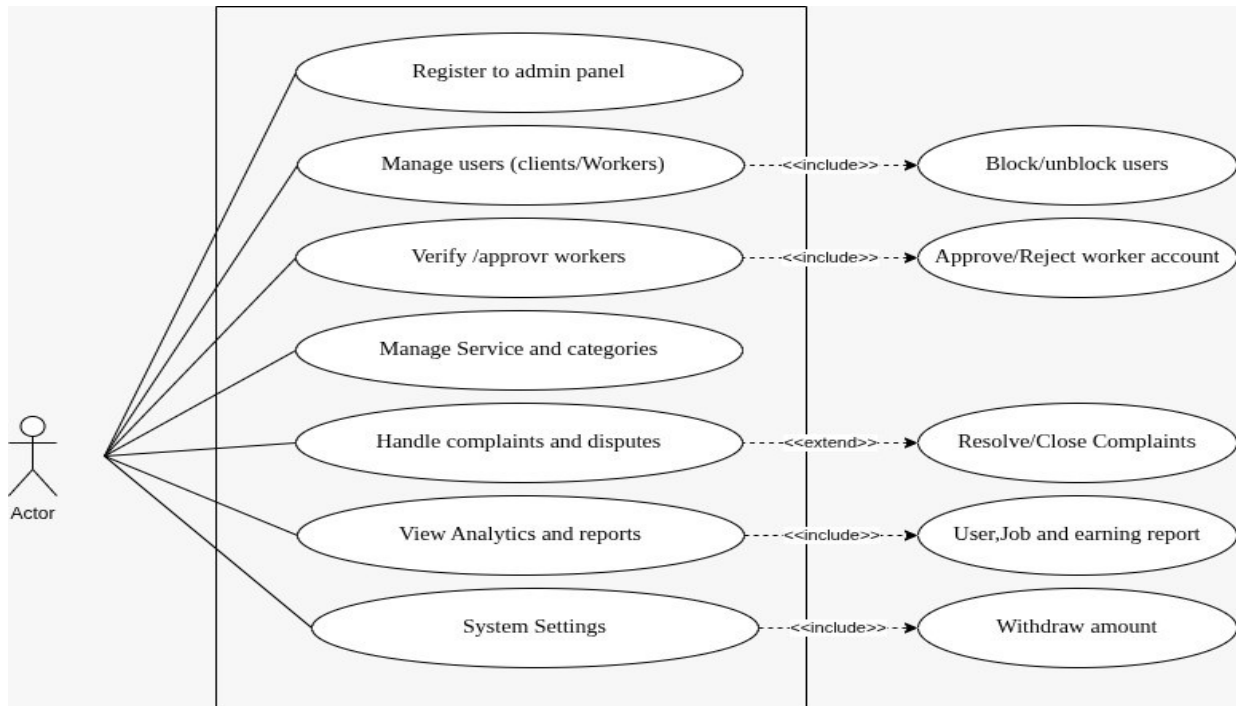
Main Success Scenario:

System analyzes job → filters workers by skill/location → ranks workers → shows best matches → sends notifications

Exception Scenario:

No workers found → system shows message → suggests alternative options

Admin(platform manager):



Admin & Monitoring Module:

Module Name: Admin & Monitoring

Primary Actor: Admin

Secondary Actor: System

Preconditions: Admin logged in

Postconditions: System managed

Trigger: Admin opens dashboard

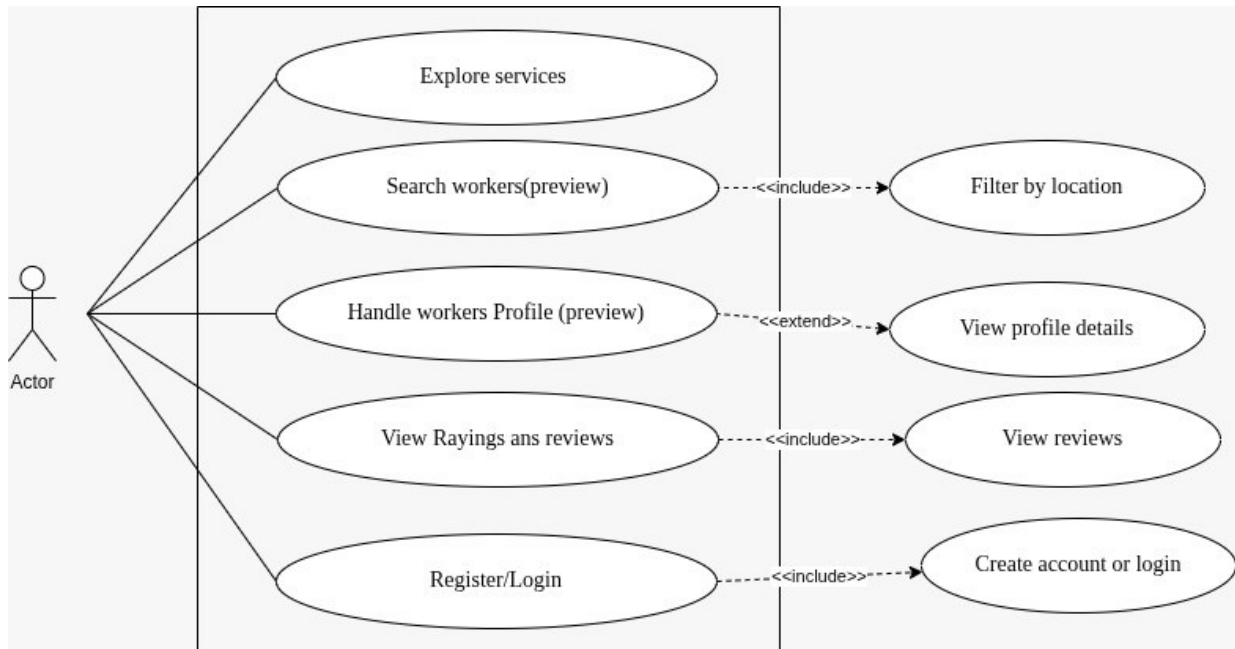
Main Success Scenario:

Admin logs in → views system → manages users → verifies workers → handles complaints → monitors system

Exception Scenario:

Unauthorized access → access denied → system logs issue → admin retries login

Guest User:



Worker Managemant Module :

Module Name: Worker Management

Primary Actor: Worker

Secondary Actor: Admin

Preconditions: Worker registered

Postconditions: Profile updated and verified

Trigger: Worker updates profile

Main Success Scenario:

Worker logs in → adds skills and details → submits → system saves → admin verifies → profile becomes active

Exception Scenario:

Incomplete or invalid details → admin rejects → profile marked pending → worker must update again

2.4 Operating Environment

- Android 10 or above
- Windows 10/11
- Linux Ubuntu
- Web Browsers: Chrome, Firefox, Edge
- Internet Connection Required
- MySQL Database Server
- Cloud Hosting Platform

2.5 Design and Implementation Constraints

- Must support voice recognition APIs.
- Must comply with data privacy regulations.
- Requires stable internet connectivity.
- Database must use MySQL.
- Backend developed using Spring Boot.
- Frontend developed using React or Android Studio.
- Support for regional languages.
- Secure user authentication required.

2.6 User Documentation

The following documentation shall be provided:

- User Manual
- Installation Guide
- Quick Start Guide
- FAQ Document
- Online Help System
- Admin Guide
- Video Tutorials

2.7 Assumptions and Dependencies

Assumptions

- Users possess smartphones or internet-enabled devices.
- Users have access to internet connectivity.

- Voice recognition services are available and functional.
- Workers provide accurate profile information.

Dependencies

- Google Speech-to-Text API or equivalent voice service.
- GPS/Location Services.
- SMS or Email Notification Services.
- MySQL Database.
- Cloud Hosting Infrastructure.

3. External Interface Requirements

3.1 User Interfaces

- The system shall provide a **simple and user-friendly interface**.
- The interface shall support **Customer, Worker, Admin, and Guest** users.
- The system shall provide **clear navigation menus and buttons**.
- Users shall be able to enter service requests through **voice or text**.
- Forms shall provide **input validation and error messages**.
- The interface shall be **responsive** on different screen sizes.
- The design shall maintain **consistent colors, fonts, icons, and layouts** across screens.

3.2 Hardware Interfaces

- The device shall have a **microphone** for voice-based service requests.
- The system shall support **desktop computers, laptops, tablets, and smartphones**.
- The system shall use **speakers/headphones** for audio interaction when required.
- The application shall communicate with the **server through an internet connection**.
- Data shall be exchanged using standard **HTTP/HTTPS protocols**.
- The system shall support common **web browsers** such as Chrome, Firefox, and Edge.
- The server shall communicate with the **database server** to store and retrieve application data.

3.3 Software Interfaces

- The system shall use **Next.js/React** for the web application interface.
- The system shall use **MySQL** for storing user, worker, job, payment, and review data.

- The system shall communicate between frontend and backend using **REST APIs**.
- A **speech recognition service** shall convert voice input into text for service requests.
- **HTTPS** shall be used for secure communication between the application and external services.
- User, worker, job request, payment, and review data shall be **shared between the application and database**.
- The system shall exchange **request, response, authentication, and status messages** between software components.

3.4 Communications Interfaces

- The system shall use **HTTPS** for secure communication between users and the server.
- The system shall use **REST APIs** for data exchange between frontend and backend.
- Voice requests shall be transmitted to the **speech recognition service** for processing.
- Communication messages shall use standard **JSON format**.
- The system shall use **encryption and authentication** to protect user data during transmission.

4. System Features

The functional requirements of **VoiceHire** are organized based on the major features and services provided by the system.

4.1 User Registration and Login

4.1.1 Description and Priority

Allows customers, workers, and admins to create accounts and securely log in.

Priority: High

4.1.2 Stimulus/Response Sequences

- User enters registration details → System validates and creates the account.
- User enters login details → System verifies credentials.
- Invalid details → System displays an error message.

4.1.3 Functional Requirements

- **REQ-1:** The system shall allow users to register with required details.
- **REQ-2:** The system shall authenticate users using valid login credentials.
- **REQ-3:** The system shall display an error for invalid or incomplete details.

4.2 Worker Profile Management

4.2.1 Description and Priority

Allows workers to create and manage their profiles, skills, experience, and availability.

Priority: High

4.2.2 Stimulus/Response Sequences

- Worker enters profile details → System saves the information.
- Worker updates skills or availability → System updates the profile.
- Customer views worker profile → System displays the available information.

4.2.3 Functional Requirements

- **REQ-4:** The system shall allow workers to create and update profiles.
- **REQ-5:** The system shall allow workers to add skills, experience, and availability.
- **REQ-6:** The system shall display worker profiles to customers.

4.3 Voice-Based Service Request

4.3.1 Description and Priority

Allows customers to describe their required service using voice or text.

Priority: High

4.3.2 Stimulus/Response Sequences

- Customer selects voice input → System records the request.
- System converts speech into text → Request is displayed for confirmation.
- Customer submits request → System processes it.

4.3.3 Functional Requirements

- **REQ-7:** The system shall allow customers to create service requests using voice or text.
- **REQ-8:** The system shall convert voice input into text.
- **REQ-9:** The system shall allow customers to confirm or edit the generated request.

4.4 Worker Search and Matching

4.4.1 Description and Priority

Helps customers find suitable workers based on service requirements, skills, availability, and ratings.

Priority: High

4.4.2 Stimulus/Response Sequences

- Customer submits service request → System searches available workers.
- System finds matching workers → Matching workers are displayed.
- Customer selects a worker → System displays the worker details.

4.4.3 Functional Requirements

- **REQ-10:** The system shall search workers based on required services.
- **REQ-11:** The system shall display matching workers.
- **REQ-12:** The system shall allow customers to view worker details and ratings.

4.5 Job Request Management

4.5.1 Description and Priority

Allows customers to send job requests and workers to accept or reject them.

Priority: High

4.5.2 Stimulus/Response Sequences

- Customer selects worker → Sends job request.
- Worker receives request → Accepts or rejects it.
- System updates the request status → Customer is notified.

4.5.3 Functional Requirements

- **REQ-13:** The system shall allow customers to send job requests.
- **REQ-14:** The system shall allow workers to accept or reject requests.
- **REQ-15:** The system shall update and display the current request status.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system shall load pages within **3 seconds** under normal conditions.
- Search results shall be displayed within **3 seconds**.
- Voice requests shall be processed and converted to text within a reasonable time.

5.2 Safety Requirements

- The system shall prevent unauthorized users from accessing restricted features.
- User and worker information shall be handled safely.
- The system shall provide a mechanism to report **complaints and disputes**

5.3 Security Requirements

- Users shall log in using **authenticated credentials**.
- Passwords shall be stored securely.
- User data shall be protected during transmission using **HTTPS**.
- Admin functions shall be accessible only to authorized administrators.

5.4 Software Quality Attributes

- The system shall be **easy to use and understand**.
- The application shall be **reliable and responsive**.
- The system shall be **maintainable and scalable**.
- The application shall work on **different browsers and screen sizes**.

5.5 Business Rules

- Only registered users can **request or accept jobs**.
- Workers must provide their **skills and experience** before accepting jobs.

- Only verified workers can be displayed as **approved service providers**.
- Customers can provide ratings and reviews **after completing a job**.
- Admins can manage users, workers, complaints, and disputes.

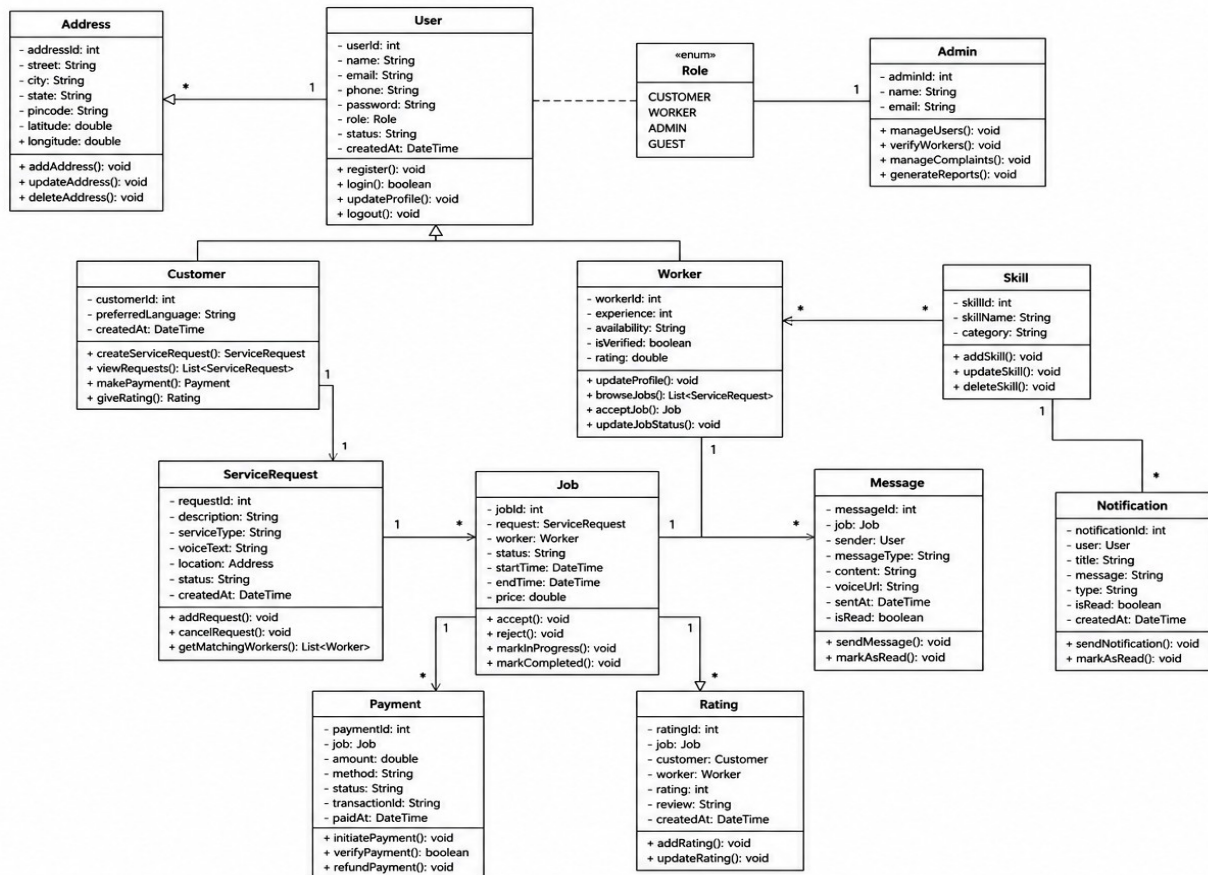
6. Other Requirements

- The application shall support **voice and text-based service requests**.
- The system shall maintain accurate records of **users, workers, jobs, payments, and reviews**.
- The system shall be accessible through **common web browsers** and different screen sizes.
- The system shall provide **regular data backup** to prevent data loss.

Appendix : Analysis Models

This appendix includes diagrams developed during system analysis and design.

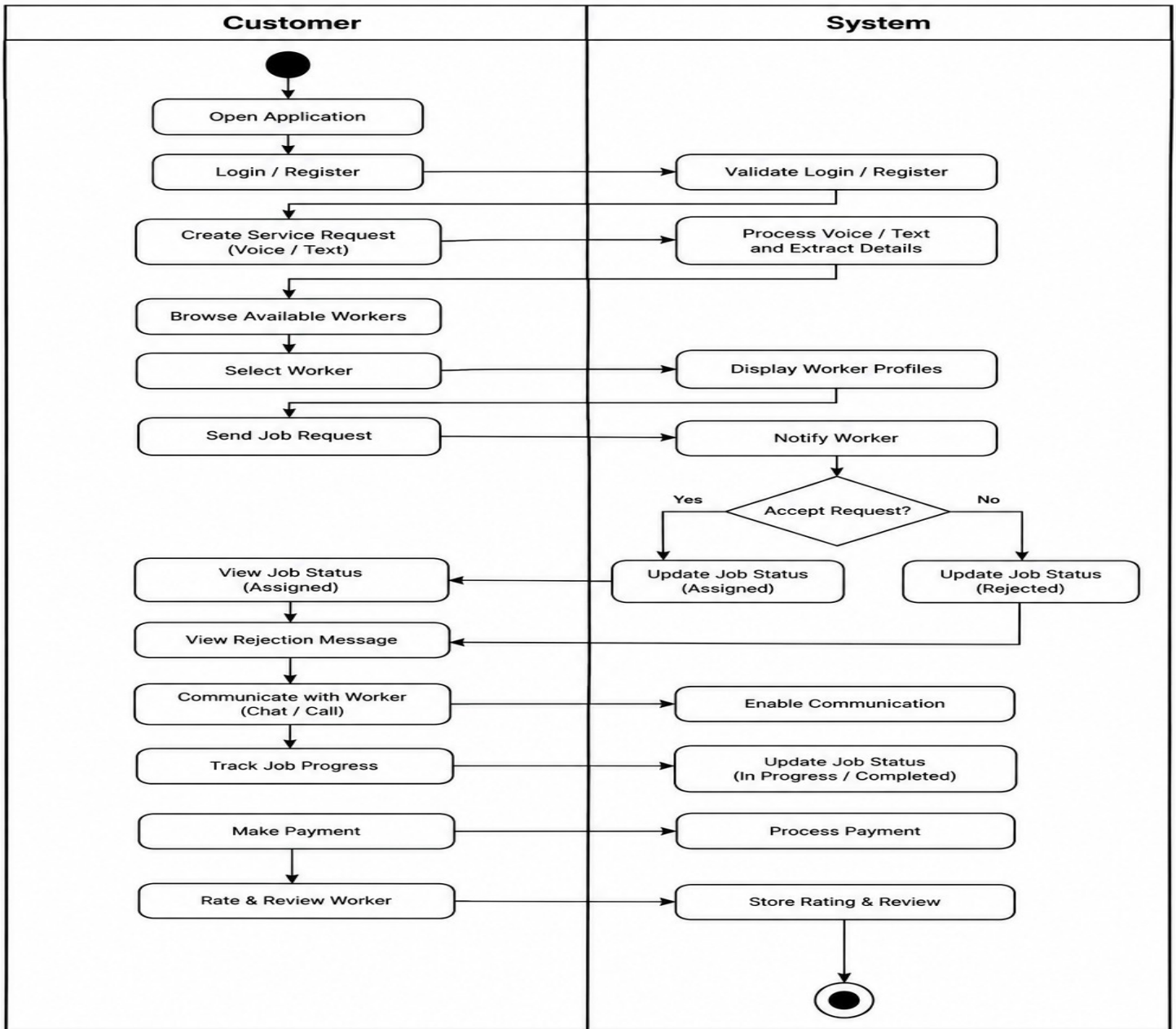
1.UML CLASS DIAGRAM:



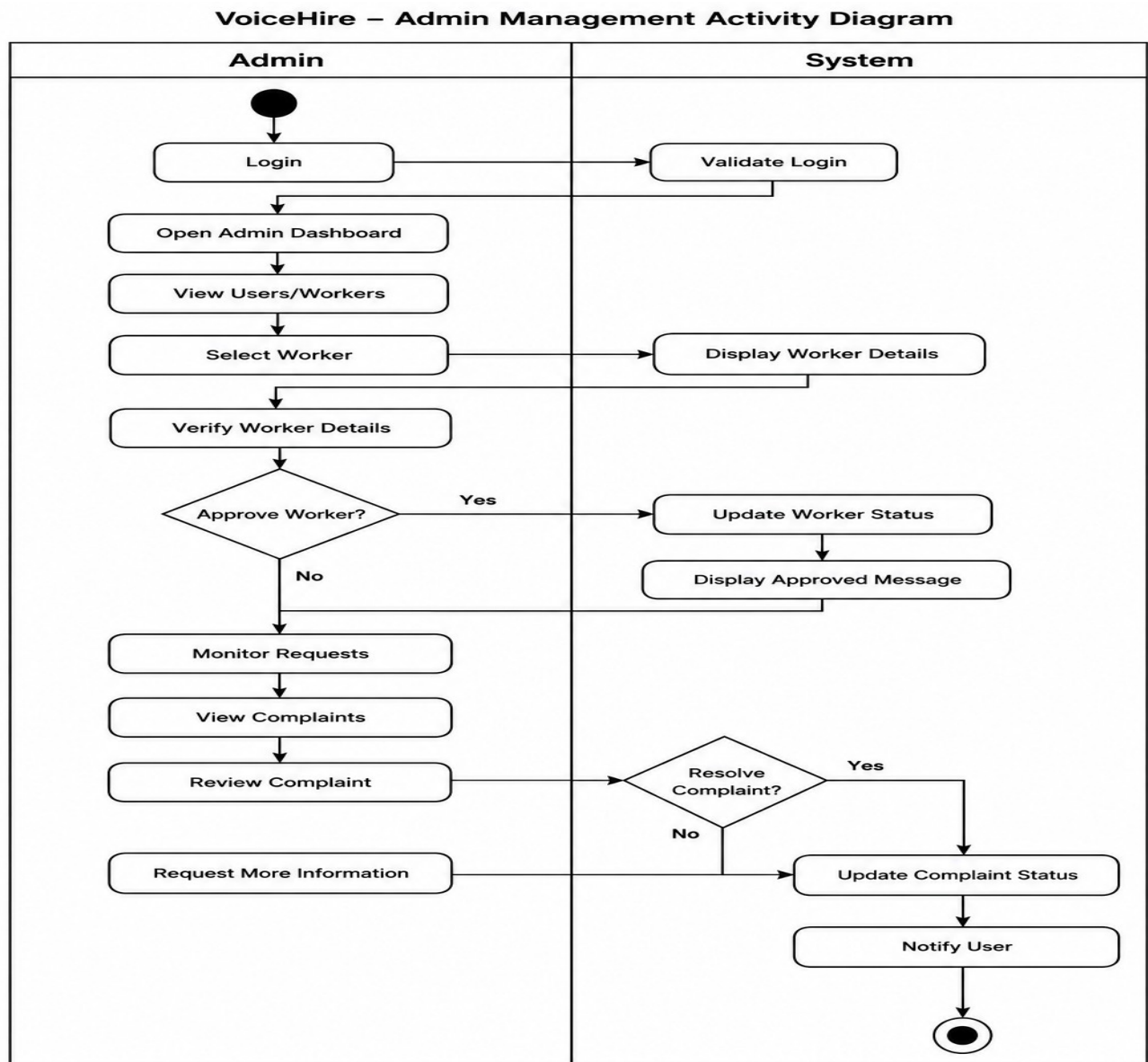
2.UML ACTIVITY DIAGRAM:

ACTIVITY DIAGRAM 1:

VoiceHire – Customer–Worker Job Hiring Activity Diagram

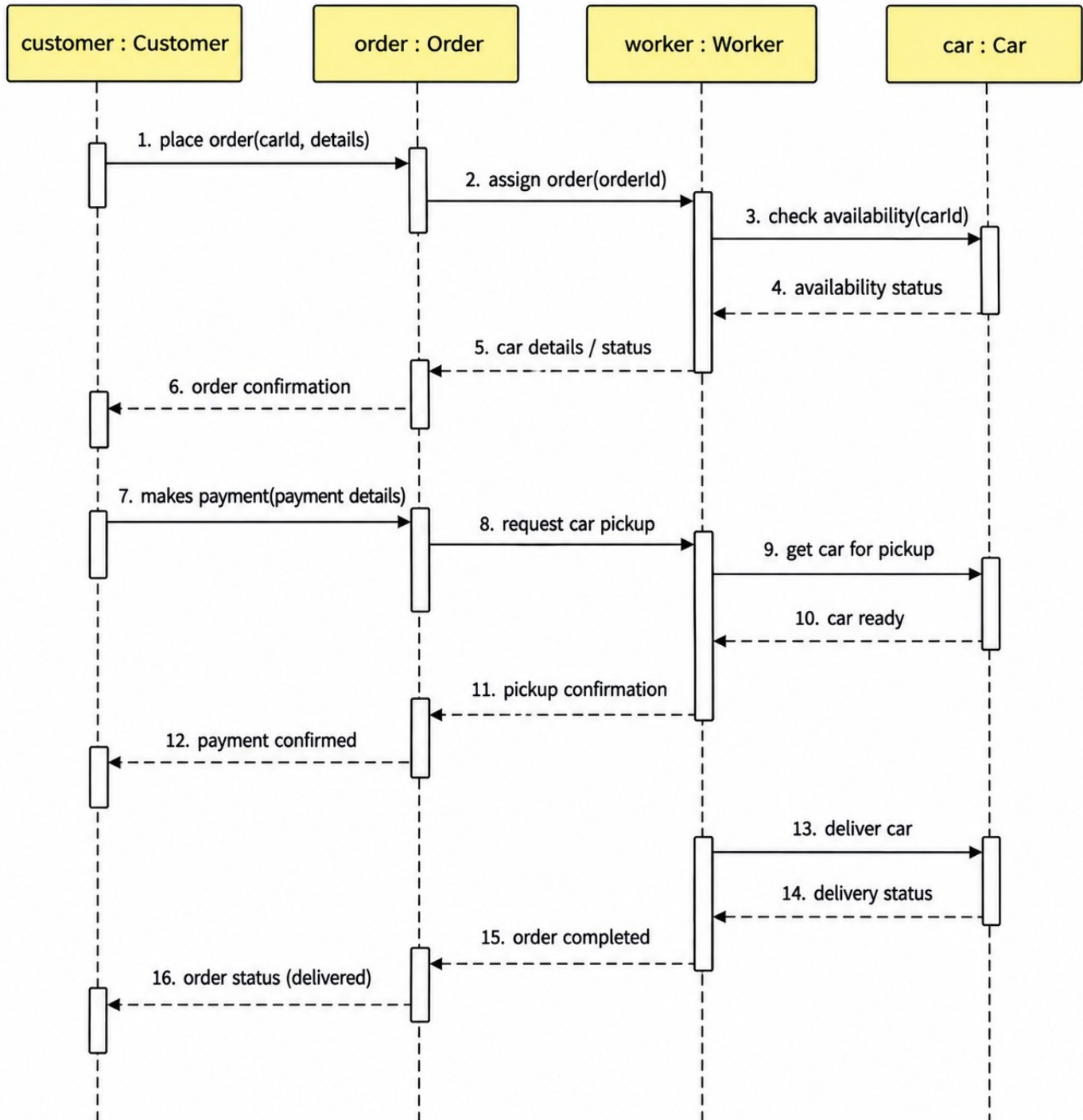


ACTIVITY DIAGRAM 2:

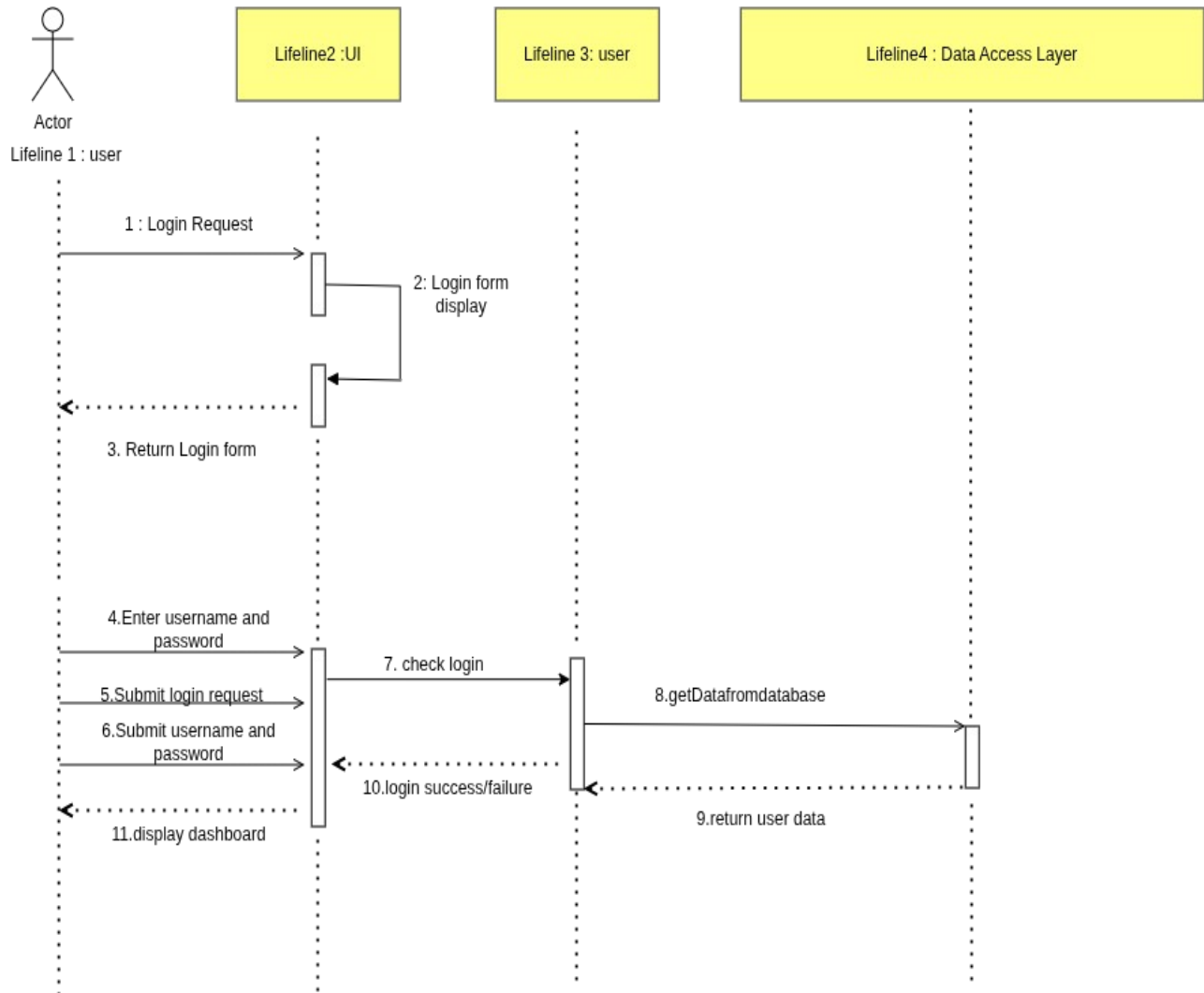


3. SEQUENCE DIAGRAM :

SEQUENCE DIAGRAM 1 : PAYMENT PROCESS

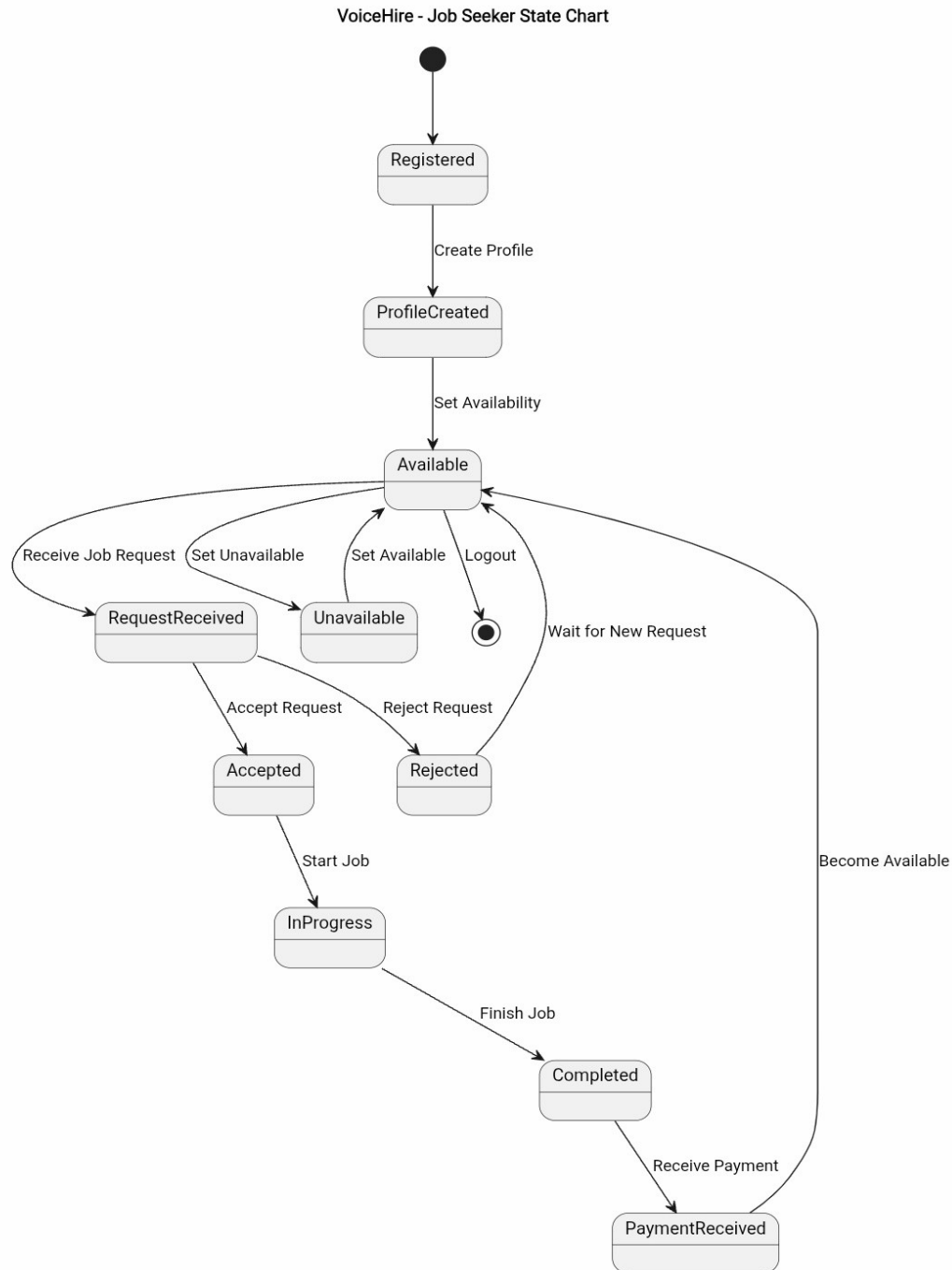


SEQUENCE DIAGRAM 2 : USER LOGIN PROCESS

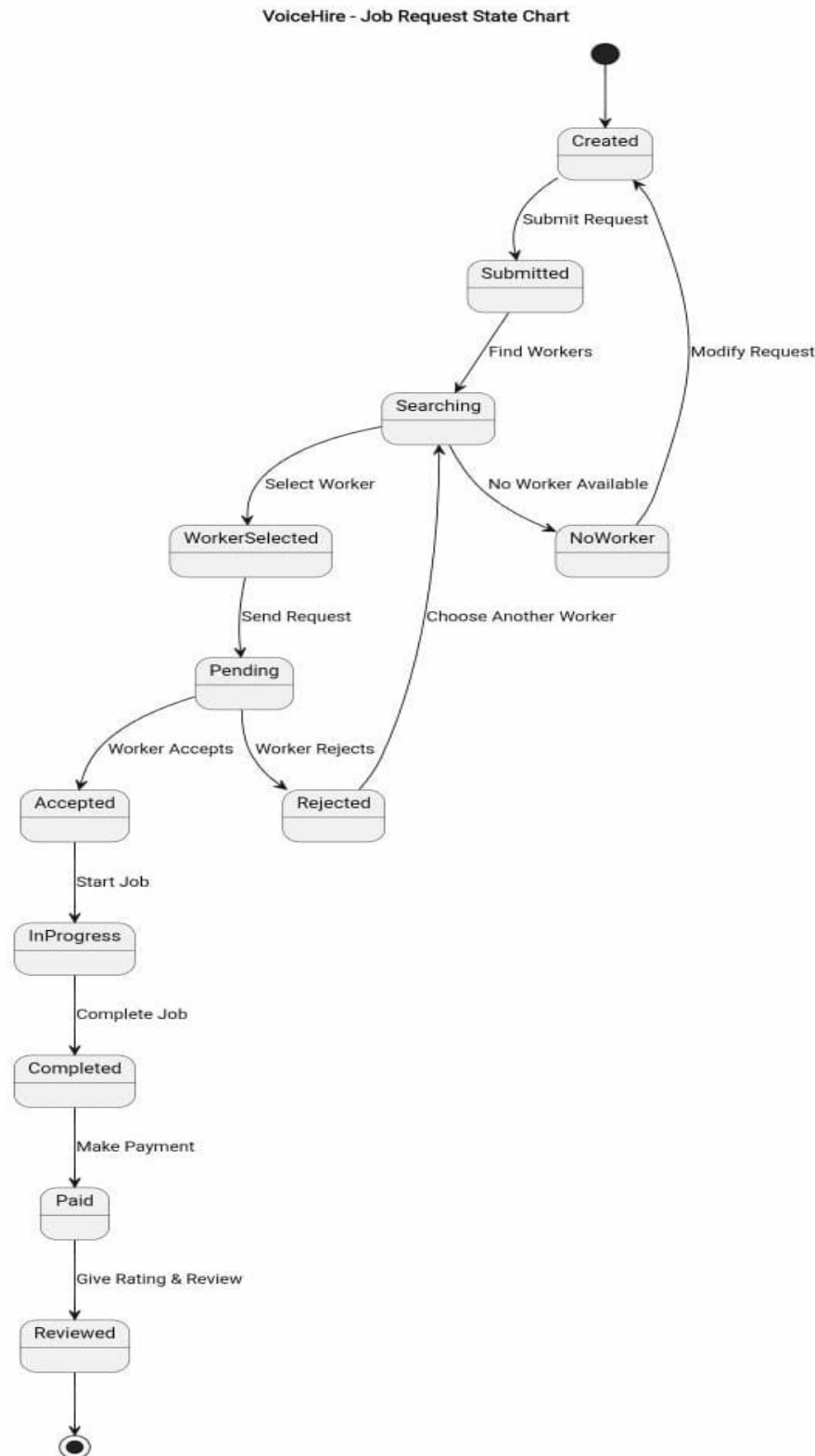


4.STATE CHART DIAGRAM :

STATE CHART DIAGRAM 1:

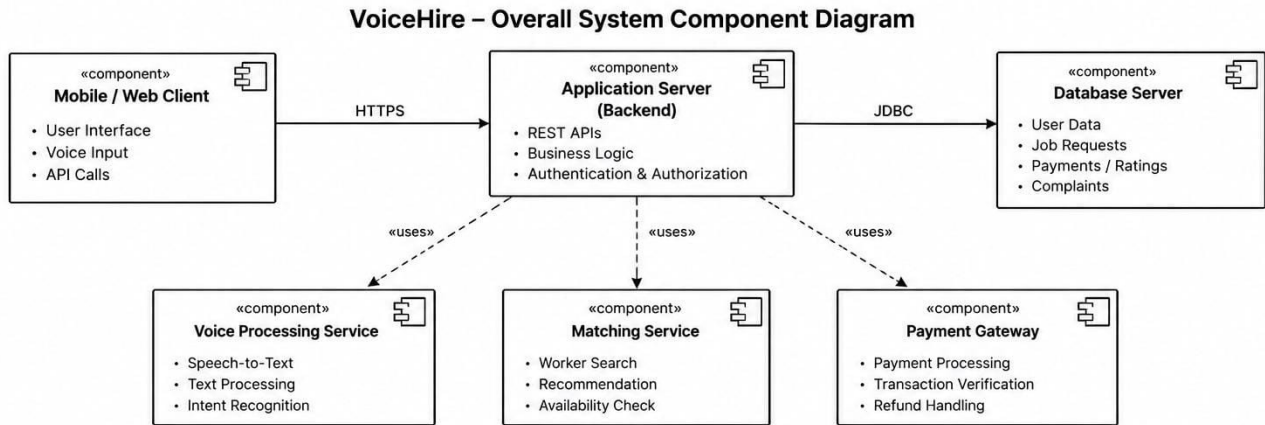


STATE CHART DIAGRAM 2:

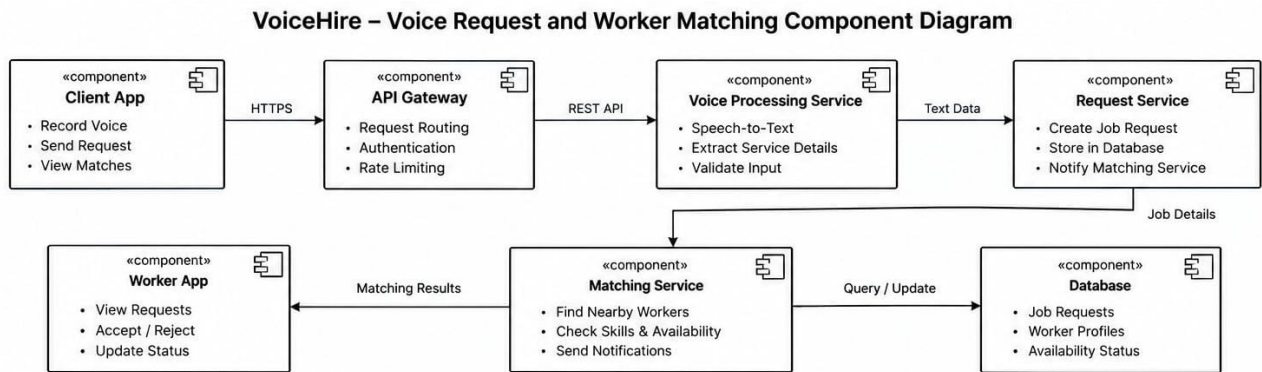


5.COMPONENT DIAGRAM :

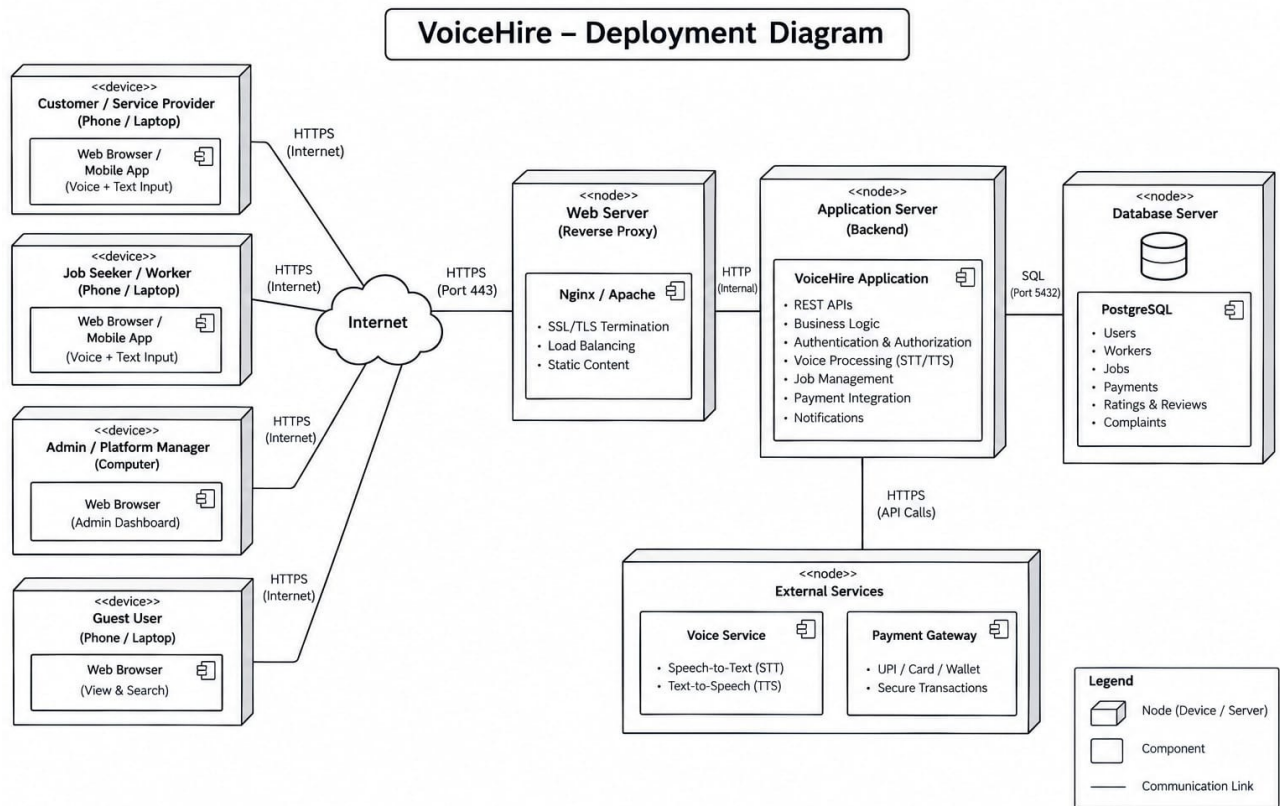
COMPONENT DIAGRAM 1:



COMPONENT DIAGRAM 2:



6.DEPLOYMENT DIAGRAM:



SOURCE CODE:

server.js:

```
const express = require('express'), fs = require('fs'), path = require('path'), crypto =
require('crypto');
const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

const DB = path.join(__dirname, 'data', 'db.json');
const hash = p => crypto.createHash('sha256').update('voicehire' + p).digest('hex');
const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2, 6);
const sessions = {};
let db;
const save = () => fs.writeFileSync(DB, JSON.stringify(db, null, 1));

// name, icon, description, keywords (English + Tamil), [male, female]
const SERVICES = [
  ['Electrician', ' ', 'Ceiling fans, wiring, light fixtures, switchboards, MCB, short circuits,
invertors.', 'electric,fan,wiring,light,switch,current,power,மின்,விளக்கு,எலக்ட்ரீசியன்', ['Ravi
Kumar', 'Subha Lakshmi']],
  ['Plumber', ' ', 'Tap repair, pipe leaks, bathroom fittings, water tank cleaning, drainage
blockage.', 'plumb,pipe,tap,leak,water,drain,tank,குழாய்,தண்ணீர்,பிளம்பர்', ['Murugan', 'Ragavi']],
  ['Carpenter', ' ', 'Furniture assembly, door lock repair, cabinet making, woodwork, bed repair.',
'carpent,wood,furniture,door,cabinet,bed,table,தச்சர்,மரம்,கதவு', ['Selvam', 'Ilamathi']],
  ['Painter', ' ', 'Full house painting, accent walls, waterproofing, exterior painting, patch
touchups.', 'paint,wall,color,colour,waterproof,பெயிண்ட்,சுவர்,வண்ணம்', ['Karthikeyan',
'Sasikala']],
  ['Cleaner', ' ', 'Deep house cleaning, sofa/carpet cleaning, bathroom and kitchen cleaning.',
'clean,sofa,carpet,wash,dust,சுத்தம்,துடை', ['Arumugam', 'Sri Devi']],
  ['Mechanic', ' ', 'Two-wheeler repair, car battery jumpstart, puncture, engine service.',
'mechanic,bike,car,vehicle,scooter,puncture,engine,மெக்கானிக்,வண்டி,பைக்', ['Senthil',
'Tamilselvi']],
  ['AC Technician', ' ', 'AC servicing, gas refilling, cooling issue fix, installation.', 'ac,air
condition,cooling,gas,ஏசி,குளிர்', ['Vignesh', 'Kaviya']],
  ['Appliance Repair', ' ', 'Washing machine repair, refrigerator fix, TV, mixer and grinder
repair.', 'washing,fridge,refrigerator,tv,mixer,grinder,appliance,oven,வாஷிங்,பிரிட்ஜ்', ['Bharathi
Raja', 'Meenakshi']],
  ['Mason', ' ', 'Brick work, plastering, tiling, flooring, wall construction and repair.',
'mason,brick,tile,plaster,floor,cement,construction,கொத்தனார்,செங்கல்', ['Palani', 'Thenmozhi']],
  ['Welder', ' ', 'Gate and grill fabrication, metal repair, roofing frames, iron work.',
'weld,gate,grill,iron,metal,steel,வெல்டிங்,இரும்பு', ['Kannan', 'Yazhini']],
  ['Gardener', ' ', 'Garden maintenance, plant care, lawn cutting, terrace garden setup.',
'garden,plant,lawn,tree,grass,தோட்டம்,செடி,மரம் வெட்டு', ['Muthu', 'Poongodi']],
  ['Driver', ' ', 'Local and outstation drivers, acting driver, goods vehicle drivers.',
'driver,drive,taxi,cab,lorry,ஓட்டுநர்,டிரைவர்', ['Gowtham', 'Nandhini']]
];
const CITIES = ['Chidambaram', 'Cuddalore', 'Chennai', 'Madurai', 'Coimbatore', 'Trichy', 'Salem',
'Pondicherry'];
const SKILLS = {
  Electrician: ['Wiring', 'Fan repair', 'MCB'], Plumber: ['Pipe leaks', 'Tap repair', 'Tank cleaning'],
```

```

Carpenter: ['Furniture', 'Door repair', 'Cabinets'], Painter: ['Interior', 'Exterior', 'Waterproofing'],
Cleaner: ['Deep cleaning', 'Sofa cleaning', 'Kitchen'], Mechanic: ['Two-wheeler', 'Car battery',
'Puncture'],
'AC Technician': ['AC service', 'Gas refill', 'Installation'], 'Appliance Repair': ['Washing
machine', 'Fridge', 'TV'],
Mason: ['Plastering', 'Tiling', 'Brick work'], Welder: ['Gates', 'Grills', 'Metal repair'],
Gardener: ['Lawn care', 'Plant care', 'Terrace garden'], Driver: ['Local trips', 'Outstation', 'Acting
driver']
};

```

```

function seed() {
  db = { users: [], workers: [], requests: [], messages: [], complaints: [] };
  const add = (name, email, role) => { const u = { id: uid(), name, email, pass:
hash('password123'), role, blocked: false }; db.users.push(u); return u; };
  add('Anand Sharma', 'anand@example.com', 'customer');
  add('Platform Manager', 'admin@example.com', 'admin');
  let n = 0;
  SERVICES.forEach(([, service, , , , pair], si) => {
    pair.forEach(([, name, gi]) => {
      const g = gi === 0 ? 'm' : 'f', u = add(name, name.split(' ')[0].toLowerCase() + '@voicehire.in',
'worker'); n++;
      db.workers.push({
        id: 'w' + n, userId: u.id, name, gender: g, service, skills: SKILLS[service], exp: 2 + (n * 3) %
12,
        rate: 250 + (n * 37) % 300, city: CITIES[n % CITIES.length], available: n % 5 !== 0,
verified: true,
        rating: 4 + ((n * 7) % 10) / 10, count: 5 + (n * 3) % 40,
        photo: `https://randomuser.me/api/portraits/${g === 'm' ? 'men' : 'women'}/${10 + n}.jpg`,
        reviews: [{ stars: 5, text: 'Very neat and on-time work.', by: 'Anand Sharma' }]
      });
    });
  });
  save();
}
if (fs.existsSync(DB)) db = JSON.parse(fs.readFileSync(DB)); else
{ fs.mkdirSync(path.dirname(DB), { recursive: true }); seed(); }

const safe = u => ({ id: u.id, name: u.name, email: u.email, role: u.role });
const fail = (res, m, c = 400) => res.status(c).json({ error: m });
const auth = (...roles) => (req, res, next) => {
  const u = db.users.find(x => x.id === sessions[(req.headers.authorization || '').replace('Bearer ',
'')]);
  if (!u) return fail(res, 'Please login first', 401);
  if (u.blocked) return fail(res, 'Account blocked by admin', 403);
  if (roles.length && !roles.includes(u.role)) return fail(res, 'Not allowed', 403);
  req.user = u; next();
};

// ----- Auth -----
app.post('/api/register', (req, res) => {
  const { name, email, password, role, gender, service, skills, exp, city, rate } = req.body;
  if (!name || !email || !password || password.length < 6) return fail(res, 'Name, email and password
(min 6 chars) required');

```

```

    if (db.users.some(u => u.email === email.toLowerCase())) return fail(res, 'Email already
registered');
    if (!['customer', 'worker'].includes(role)) return fail(res, 'Invalid role');
    const u = { id: uid(), name, email: email.toLowerCase(), pass: hash(password), role, blocked:
false };
    db.users.push(u);
    if (role === 'worker') {
      const g = gender === 'f' ? 'f' : 'm';
      db.workers.push({ id: 'w' + uid(), userId: u.id, name, gender: g, service: service || 'Electrician',
        skills: (skills || "").split(',').map(s => s.trim()).filter(Boolean), exp: +exp || 0, rate: +rate || 300,
city: city || 'Chidambaram',
        available: true, verified: false, rating: 0, count: 0, reviews: [],
        photo: `https://randomuser.me/api/portraits/${g === 'm' ? 'men' : 'women'}/${
Math.floor(Math.random() * 60) + 60}.jpg` });
    }
    save(); res.json({ ok: true });
  });
  app.post('/api/login', (req, res) => {
    const { email, password, role } = req.body;
    const u = db.users.find(x => x.email === (email || "").toLowerCase() && x.pass ===
hash(password || ""));
    if (!u || (role && u.role !== role)) return fail(res, 'Invalid credentials for ' + (role || 'account'));
    if (u.blocked) return fail(res, 'Account blocked by admin', 403);
    const t = uid() + uid(); sessions[t] = u.id; res.json({ token: t, user: safe(u) });
  });

// ----- Public: services & workers (guest allowed) -----
const visible = () => db.workers.filter(w => w.verified && !db.users.find(u => u.id ===
w.userId).blocked);
app.get('/api/services', (req, res) => res.json(SERVICES.map(([name, icon, desc]) => ({ name,
icon, desc, count: visible().filter(w => w.service === name).length })));
app.get('/api/workers', (req, res) => res.json(visible().filter(w => !req.query.service || w.service
=== req.query.service)));
app.get('/api/workers/:id', (req, res) => { const w = db.workers.find(x => x.id === req.params.id);
w ? res.json(w) : fail(res, 'Worker not found', 404); });

// Voice/text -> service detection (speech-to-text happens in browser)
app.post('/api/voice-parse', (req, res) => {
  const t = (req.body.text || "").toLowerCase();
  let best = null, score = 0;
  SERVICES.forEach(s => { const sc = s[3].split(',').filter(k => t.includes(k.trim())).length; if (sc >
score) { score = sc; best = s[0]; } });
  res.json({ service: best, workers: best ? visible().filter(w => w.service === best) : [] });
});

// ----- Worker self -----
app.get('/api/me/worker', auth('worker'), (req, res) => res.json(db.workers.find(w => w.userId ===
req.user.id)));
app.post('/api/me/worker/toggle', auth('worker'), (req, res) => { const w = db.workers.find(x =>
x.userId === req.user.id); w.available = !w.available; save(); res.json(w); });

// ----- Job requests -----
const decorate = r => { const w = db.workers.find(x => x.id === r.workerId), c = db.users.find(x
=> x.id === r.customerId); return { ...r, workerName: w.name, customerName: c.name }; };

```

```
app.post('/api/requests', auth('customer'), (req, res) => {
  const w = db.workers.find(x => x.id === req.body.workerId);
  if (!w || !req.body.description) return fail(res, 'Worker and job description required');
  const r = { id: 'r' + uid(), customerId: req.user.id, workerId: w.id, service: w.service, description:
req.body.description,
  date: req.body.date || '', address: req.body.address || '', status: 'pending', amount: 0, rated: false,
created: new Date().toISOString() };
  db.requests.push(r); save(); res.json(r);
});
app.get('/api/requests', auth(), (req, res) => {
  const me = db.workers.find(w => w.userId === req.user.id);
  const list = db.requests.filter(r => req.user.role === 'admin' || r.customerId === req.user.id || (me
&& r.workerId === me.id));
  res.json(list.reverse().map(decorate));
});
app.patch('/api/requests/:id', auth('worker', 'customer'), (req, res) => {
  const r = db.requests.find(x => x.id === req.params.id); if (!r) return fail(res, 'Not found', 404);
  const w = db.workers.find(x => x.id === r.workerId), a = req.body.action;
  const isW = req.user.role === 'worker' && w.userId === req.user.id, isC = req.user.role ===
'customer' && r.customerId === req.user.id;
  if (!isW && !isC) return fail(res, 'Not your request', 403);
  const T = { accept: ['pending', 'accepted'], reject: ['pending', 'rejected'], start: ['accepted',
'in_progress'], complete: ['in_progress', 'completed'] };
  if (isW && T[a]) {
    if (r.status !== T[a][0]) return fail(res, 'Invalid step');
    r.status = T[a][1];
    if (a === 'complete') r.amount = Math.max(1, +req.body.hours || 2) * w.rate;
  } else if (isC && a === 'pay') {
    if (r.status !== 'completed') return fail(res, 'Job not completed yet');
    r.status = 'paid'; r.paidAt = new Date().toISOString(); // simulated payment
  } else if (isC && a === 'rate') {
    const s = +req.body.stars;
    if (r.status !== 'paid' || r.rated || s < 1 || s > 5) return fail(res, 'Cannot rate now');
    w.rating = (w.rating * w.count + s) / (w.count + 1); w.count++;
    w.reviews.unshift({ stars: s, text: req.body.text || '', by: req.user.name }); r.rated = true;
  } else return fail(res, 'Invalid action');
  save(); res.json(decorate(r));
});
function canChat(u, r) { const w = db.workers.find(x => x.id === r.workerId); return r &&
(r.customerId === u.id || w.userId === u.id || u.role === 'admin'); }
app.get('/api/messages/:rid', auth(), (req, res) => {
  const r = db.requests.find(x => x.id === req.params.rid); if (!canChat(req.user, r)) return fail(res,
'Not allowed', 403);
  res.json(db.messages.filter(m => m.rid === r.id));
});
app.post('/api/messages/:rid', auth(), (req, res) => {
  const r = db.requests.find(x => x.id === req.params.rid); if (!canChat(req.user, r) || !
req.body.text) return fail(res, 'Not allowed', 403);
  db.messages.push({ rid: r.id, from: req.user.id, name: req.user.name, text: req.body.text, at: new
Date().toISOString() }); save(); res.json({ ok: true });
});
app.post('/api/complaints', auth('customer', 'worker'), (req, res) => {
```

```
db.complaints.push({ id: 'c' + uid(), by: req.user.name, requestId: req.body.requestId, text:
req.body.text, status: 'open' }); save(); res.json({ ok: true });
});app.get('/api/admin/overview', auth('admin'), (req, res) => res.json({
  users: db.users.filter(u => u.role !== 'admin').map(u => ({ ...safe(u), blocked: u.blocked })),
  workers: db.workers, requests: db.requests.length, complaints: db.complaints
}));app.patch('/api/admin/workers/:id', auth('admin'), (req, res) => { const w = db.workers.find(x
=> x.id === req.params.id); w.verified = !w.verified; save(); res.json(w); });
app.patch('/api/admin/users/:id', auth('admin'), (req, res) => { const u = db.users.find(x => x.id
=== req.params.id); u.blocked = !u.blocked; save(); res.json(safe(u)); });
app.patch('/api/admin/complaints/:id', auth('admin'), (req, res) => { const c = db.complaints.find(x
=> x.id === req.params.id); c.status = 'resolved'; save(); res.json(c); });
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`VoiceHire running at http://localhost:${PORT}`));
```

index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>VoiceHire – Voice-First Services</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header id="nav"></header>
<main id="app"></main>
<div id="toast"></div>
<script src="app.js"></script>
</body>
</html>
```

package.json:

```
{
  "name": "voicehire",
  "version": "1.0.0",
  "description": "Voice-enabled platform connecting customers with skilled blue-collar workers",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2" }
}
```

app.js:

```

const $ = s => document.querySelector(s), app = $('#app');

let S = { token: localStorage.vhT || '', user: JSON.parse(localStorage.vhU || 'null'), lang:
localStorage.vhL || 'en-IN' };

const esc = s => String(s ?? "").replace(/[\&<>"/]/g, c => ({ '&': '&', '<': '<', '>': '>', '"': '"', "'": "'" }
[c]));

const toast = m => { const t = $('#toast'); t.textContent = m; t.classList.add('show'); setTimeout(()
=> t.classList.remove('show'), 2800); };

const act = f => async (...a) => { try { await f(...a); } catch (e) { toast(e.message); } };

async function api(url, method = 'GET', body) {

const r = await fetch('/api' + url, { method, headers: { 'Content-Type': 'application/json',
Authorization: 'Bearer ' + S.token }, body: body ? JSON.stringify(body) : undefined });

const d = await r.json().catch(() => ({}));

if (!r.ok) throw new Error(d.error || 'Something went wrong');

return d;

}

const stars = r => '★'.repeat(Math.round(r)) + '☆'.repeat(5 - Math.round(r));

const wcard = w => `<a class="card wc" href="#/worker/${w.id}"><div><h3>${esc(w.name)} ${w.verified ? ' ' : ''}</h3><p class="mut">${
w.service} · ${w.city}</p><p class="star">${stars(w.rating)} <small>${w.rating.toFixed(1)} ($
{w.count})</small></p><p><span class="tag ${w.available ? 'ok' : 'no'}">${w.available ?
'Available' : 'Busy'}</span> ₹${w.rate}/hr · ${w.exp} yrs</p></div></a>`;

function nav() {

const h = location.hash || '#/';

$('#nav').innerHTML = `<a class="brand" href="#/"><span
class="logo">    </span><span><b>VOICEHIRE</b><small>VOICE-FIRST
SERVICES</small></span></a>

<nav>${[[['#/', 'Home'], ['#/services', 'Services'], ['#/workers', 'Find Workers']].map(([l, t]) => `<a
href="${l}" class="${h === l ? 'on' : ''}">${t}</a>`)).join("")}<a href="#/voice" class="btn">
Voice Request</a></nav>

```

```
<div class="right"><select id="lang"><option value="en-IN">English</option><option value="ta-IN">தமிழ்</option></select>
```

```
${S.user ? `<a href="#/dashboard">    ${esc(S.user.name.split(' ')[0])}</a><a href="#" id="out">Logout</a>` : `<a href="#/login">Login</a><a href="#/register" class="btn">Register</a>`} </div>`;
```

```
$('#lang').value = S.lang;
```

```
$('#lang').onchange = e => { S.lang = localStorage.vhL = e.target.value; };
```

```
const o = $('#out');
```

```
if (o) o.onclick = e => { e.preventDefault(); S.token = ''; S.user = null; localStorage.removeItem('vhT'); localStorage.removeItem('vhU'); location.hash = '#/'; nav(); };
```

```
/* ----- Voice ----- */
```

```
function voiceBox() {
```

```
return `<div class="card voice"><span class="pill">    Multilingual Voice AI Recognizer</span><h2>Tell us what you need</h2>
```

```
<p class="mut">Click the microphone button and describe your service requirement naturally.</p>
```

```
<button id="mic" class="mic">    </button><p id="vstat" class="mut">Tap to speak (${S.lang === 'ta-IN' ? 'Tamil' : 'English'})</p>
```

```
<textarea id="vtxt" rows="3" placeholder="e.g. My ceiling fan is not working / என் வீட்டில் குழாய் கசிகிறது"></textarea>
```

```
<button id="vgo" class="btn big">Find Workers</button><div id="vres"></div></div>`;
```

```
}
```

```
function voiceInit() {
```

```
const R = window.SpeechRecognition || window.webkitSpeechRecognition;
```

```
const go = act(async () => {
```

```
const t = $('#vtxt').value.trim(); if (!t) return toast('Speak or type your requirement first');
```

```
const d = await api('/voice-parse', 'POST', { text: t });
```

```
$('#vres').innerHTML = d.service
```

```
? `

```
const r = new R(); r.lang = S.lang;

r.onstart = () => { $('#mic').classList.add('rec'); $('#vstat').textContent = 'Listening...'; };

r.onend = () => { $('#mic').classList.remove('rec'); $('#vstat').textContent = 'Tap to speak'; };

r.onresult = e => { $('#vtxt').value = e.results[0][0].transcript; go(); };

r.onerror = () => toast('Could not hear you. Please try again.');
```



```
r.start();

};

$('#vgo').onclick = go;

}

/* ----- Pages ----- */

function home() {

app.innerHTML = `
```


```



```
app.innerHTML = `

# 


```

```
<input id="q" class="search" placeholder="    Search service name or task..."><div class="grid4" id="sg"></div>`;
```

```
const draw = () => { const q = $('#q').value.toLowerCase(); $('#sg').innerHTML = s.filter(x => (x.name + x.desc).toLowerCase().includes(q)).map(x => `
```

```
async function workers() {
```

```
const sv = new URLSearchParams(location.hash.split('?')[1] || "").get('service') || "";
```

```
const list = await api('/workers?service=' + encodeURIComponent(sv));
```

```
app.innerHTML = `

# 


```

```
const draw = () => { const q = $('#q').value.toLowerCase(); $('#wl').innerHTML = list.filter(w => (w.name + w.skills.join() + w.city).toLowerCase().includes(q)).map(wcard).join("") || '<p class="mut">No workers found.</p>'; };

$('#q').oninput = draw; draw();

}
```

```
async function workerPage(id) {
```

```
const w = await api('/workers/' + id);
```

```
app.innerHTML = `

```
<p class="mut">${w.service} · ${esc(w.city)} · ${w.exp} yrs experience · ₹${w.rate}/hr</p><p class="star">${stars(w.rating)} ${w.rating.toFixed(1)} (${w.count} reviews)</p>
```



```
<p>${w.skills.map(s => `
```


```

```
<div class="card"><h2>Reviews</h2>${w.reviews.map(r => `<p><span class="star">${
  stars(r.stars)}</span> ${esc(r.text)} <small class="mut">— ${esc(r.by)}</small></p>`).join("")
|| '<p class="mut">No reviews yet.</p>' }</div>
```

```
${S.user?.role === 'customer' ? `<div class="card"><h2>Send Job Request</h2><textarea id="d"
rows="3" placeholder="Describe the work..."></textarea><input id="dt" type="date"><input
id="ad" placeholder="Address"><button id="send" class="btn big">Send
Request</button></div>`
```

```
: `<p class="mut c">${S.user ? 'Only customers can send job requests.' : '<a
href="#/login"><u>Login</u></a> or <a href="#/register"><u>register</u></a> to send a job
request.' }</p>` `;
```

```
const b = $('#send');
```

```
if (b) b.onclick = act(async () => { await api('/requests', 'POST', { workerId: id, description: $
('#d').value, date: $('#dt').value, address: $('#ad').value }); toast('Request sent!'); location.hash =
'#/dashboard'; });
```

```
}
```

```
/* ----- Auth ----- */
```

```
async function doLogin(email, password, role) {
```

```
const d = await api('/login', 'POST', { email, password, role });
```

```
S.token = localStorage.vhT = d.token; S.user = d.user; localStorage.vhU = JSON.stringify(d.user);
```

```
location.hash = '#/dashboard'; nav();
```

```
}
```

```
function login() {
```

```
let role = 'customer';
```

```
const demo = [['Customer Demo: Anand Sharma', 'anand@example.com', 'customer'], ['Worker
Demo: Ravi Kumar (Electrician)', 'ravi@voicehire.in', 'worker'], ['Admin Demo: Platform
Manager', 'admin@example.com', 'admin']];
```

```
app.innerHTML = `<div class="card auth"><div class="ico c"> </div><h2 class="c">Login to
VoiceHire</h2><p class="mut c">Select your account type or use 1-click academic demo logins
below.</p>
```

```
<div class="tabs">${['customer', 'worker', 'admin'].map(r => `<button data-r="${r}" class="${r
=== role ? 'on' : ''}>${r[0].toUpperCase() + r.slice(1)}</button>`).join("")}</div>
```

```
<label>Email Address</label><input id="em"
placeholder="anand@example.com"><label>Password</label><input id="pw" type="password"
placeholder="password123">
```

```
<button id="go" class="btn big w">Login as customer</button><h4 class="c"> QUICK 1-
CLICK ACADEMIC DEMO LOGIN</h4>
```

```
${demo.map((d, i) => `<button class="demo" data-i="${i}">${d[0]} <small>${d[2]
[0].toUpperCase() + d[2].slice(1)} →</small></button>`).join("")}</div>`;
```

```
document.querySelectorAll('.tabs button').forEach(b => b.onclick = () => { role = b.dataset.r;
document.querySelectorAll('.tabs button').forEach(x => x.classList.toggle('on', x === b)); $
('#go').textContent = 'Login as ' + role; });
```

```
$('#go').onclick = act(() => doLogin($('#em').value, $('#pw').value, role));
```

```
document.querySelectorAll('.demo').forEach(b => b.onclick = act(() => { const d =
demo[b.dataset.i]; return doLogin(d[1], 'password123', d[2]); }));
```

```
}
```

```
async function register() {
```

```
const sv = await api('/services');
```

```
app.innerHTML = `<div class="card auth"><h2 class="c">Create your VoiceHire account</h2>
```

```
<select id="role"><option value="customer">I need workers (Customer)</option><option
value="worker">I am a worker (Job Seeker)</option></select>
```

```
<input id="nm" placeholder="Full name"><input id="em" placeholder="Email"><input id="pw"
type="password" placeholder="Password (min 6 chars)">
```

```
<div id="wx" style="display:none"><select id="gn"><option value="m">Male</option><option
value="f">Female</option></select><select id="sv">${sv.map(s =>
`<option>${s.name}</option>`).join("")}</select>
```

```
<input id="sk" placeholder="Skills (comma separated)"><input id="ex" type="number"
placeholder="Years of experience"><input id="ct" placeholder="City"><input id="rt"
type="number" placeholder="Rate per hour (₹)"></div>
```

```
<button id="go" class="btn big w">Register</button></div>`;
```

```
$('#role').onchange = e => $('#wx').style.display = e.target.value === 'worker' ? 'block' : 'none';
```

```
$('#go').onclick = act(async () => {
```

```
const role = $('#role').value;
```

```
await api('/register', 'POST', { role, name: $('#nm').value, email: $('#em').value, password: $
($('#pw').value, gender: $('#gn').value, service: $('#sv').value, skills: $('#sk').value, exp: $
($('#ex').value, city: $('#ct').value, rate: $('#rt').value });

toast(role === 'worker' ? 'Registered! Admin will verify your profile.' : 'Registered! Please login.');
```

location.hash = '#/login';

```
});

}

/* ----- Dashboards ----- */

function reqCard(r) {

const w = S.user.role === 'worker', b = [];

if (w) { if (r.status === 'pending') b.push(['accept', 'Accept'], ['reject', 'Reject']); if (r.status ===
'accepted') b.push(['start', 'Start Job']); if (r.status === 'in_progress') b.push(['complete', 'Mark
Completed']); }

else { if (r.status === 'completed') b.push(['pay', `Pay ₹${r.amount}`]); if (r.status === 'paid'
&& !r.rated) b.push(['rate', 'Rate & Review']); }

if (['accepted', 'in_progress', 'completed', 'paid'].includes(r.status)) b.push(['chat', 'Chat']);

b.push(['report', 'Report']);

return `
```

```

extra = `<div class="card req"><span>Availability: <span class="tag ${ me.available ? 'ok' :
'no' }">${ me.available ? 'Available' : 'Busy' }</span> ${ me.verified ? '' : '<span class="tag
no">Awaiting admin verification</span>'}</span><button class="btn dark sm" id="av">Toggle
availability</button></div>`;

}

app.innerHTML = `<h1>${ S.user.role === 'worker' ? 'Worker' : 'Customer' } Dashboard</h1>${
{extra}}${ S.user.role === 'customer' ? '<a class="btn" href="#/voice">    New Voice Request</a>'
: ''}<div id="rl">${ rs.map(reqCard).join("") } || '<p class="mut">No job requests yet.</p>'</div>`;

const av = $('#av'); if (av) av.onclick = act(async () => { await api('/me/worker/toggle', 'POST');
dash(); });

$('#rl').onclick = act(async e => {

const b = e.target.closest('[data-a]'); if (!b) return; const { a, id } = b.dataset;

if (a === 'chat') return location.hash = '#/chat/' + id;

if (a === 'report') { const t = prompt('Describe your complaint'); if (t) { await api('/complaints',
'POST', { requestId: id, text: t }); toast('Complaint sent to admin'); } return; }

const body = { action: a };

if (a === 'complete') body.hours = +prompt('Hours worked?', '2') || 2;

if (a === 'rate') { body.stars = +prompt('Rating (1-5)', '5'); body.text = prompt('Write a short
review') || ''; }

await api('/requests/' + id, 'PATCH', body); toast('Updated'); dash();

});

}

async function adminView() {

const d = await api('/admin/overview');

app.innerHTML = `<h1>Admin Panel</h1><div class="stats"><div class="card"><b>${
{d.users.length}}</b><p>Users</p></div><div class="card"><b>${d.workers.filter(w => !
w.verified).length}</b><p>Pending workers</p></div><div
class="card"><b>${d.requests}</b><p>Job requests</p></div><div class="card"><b>${
{d.complaints.filter(c => c.status === 'open').length}}</b><p>Open complaints</p></div></div>

<h2>Workers</h2>${ d.workers.map(w => `<div class="card req"><span> <b>${esc(w.name)}</b> · $
{w.service} <span class="tag ${w.verified ? 'ok' : 'no'}">${w.verified ? 'Verified' :

```

```
'Pending' }</span></span><button class="btn dark sm" data-w="\${w.id}">\${w.verified ?
'Revoke' : 'Verify'}</button></div>`).join("")}

<h2>Users</h2>\${d.users.filter(u => u.role === 'customer').map(u => `<div class="card
req"><span>\${esc(u.name)} · \${esc(u.email)} \${u.blocked ? '<span class="tag
no">Blocked</span>' : ''}</span><button class="btn dark sm" data-u="\${u.id}">\${u.blocked ?
'Unblock' : 'Block'}</button></div>`).join("")}

<h2>Complaints</h2>\${d.complaints.map(c => `<div class="card req"><span>\${esc(c.by)}: \${
{esc(c.text)} <span class="tag st-\${c.status === 'open' ? 'pending' :
'paid'}">\${c.status}</span></span>\${c.status === 'open' ? `<button class="btn dark sm" data-
c="\${c.id}">Resolve</button>` : ''}</div>`).join("") || '<p class="mut">No complaints.</p>'}`;

app.onclick = act(async e => {

const b = e.target.closest('button[data-w],button[data-u],button[data-c]'); if (!b) return;

if (b.dataset.w) await api('/admin/workers/' + b.dataset.w, 'PATCH');

if (b.dataset.u) await api('/admin/users/' + b.dataset.u, 'PATCH');

if (b.dataset.c) await api('/admin/complaints/' + b.dataset.c, 'PATCH');

adminView();

});

}

const routes = [/^#Vservices$/, services], [/^#Vworkers(?:.*)?$/, workers], [/^#VworkerV(\w+)$/,
workerPage], [/^#Vvoice$/, voice], [/^#Vlogin$/, login], [/^#Vregister$/, register],
[/^#Vdashboard$/, dash], [/^#VchatV(\w+)$/, chat]];

async function route() {

nav(); app.onclick = null; const h = location.hash || '#/';

for (const [re, fn] of routes) { const m = h.match(re); if (m) { try { return await fn(m[1]); } catch
(e) { app.innerHTML = `<p class="err">\${esc(e.message)}</p>`; return; } } }

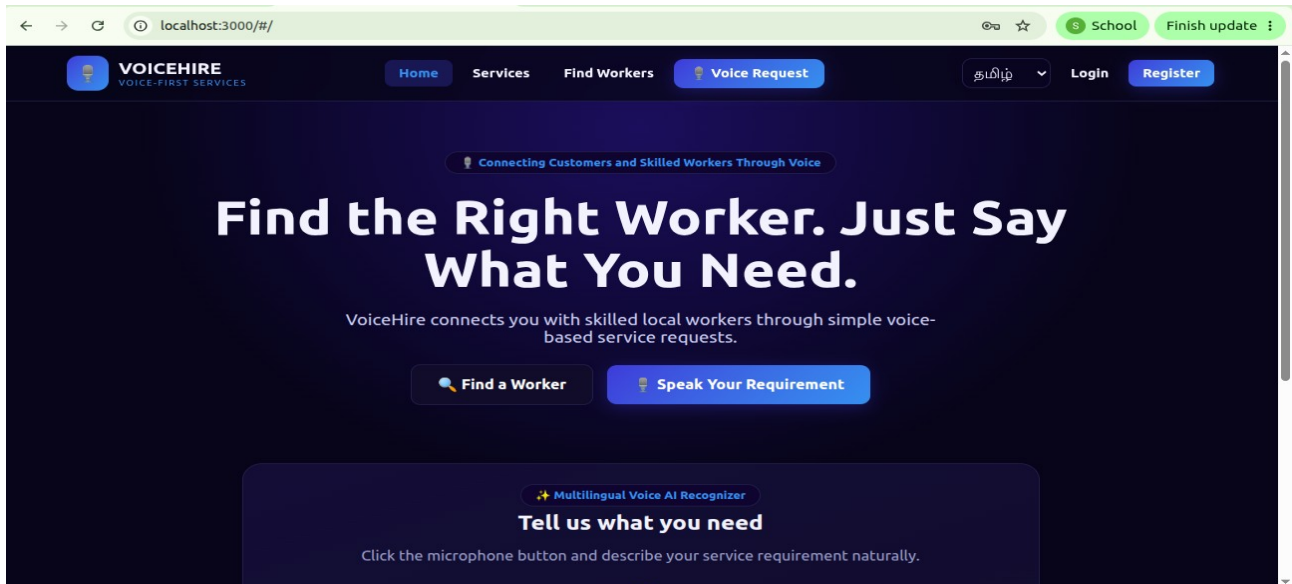
home();

}

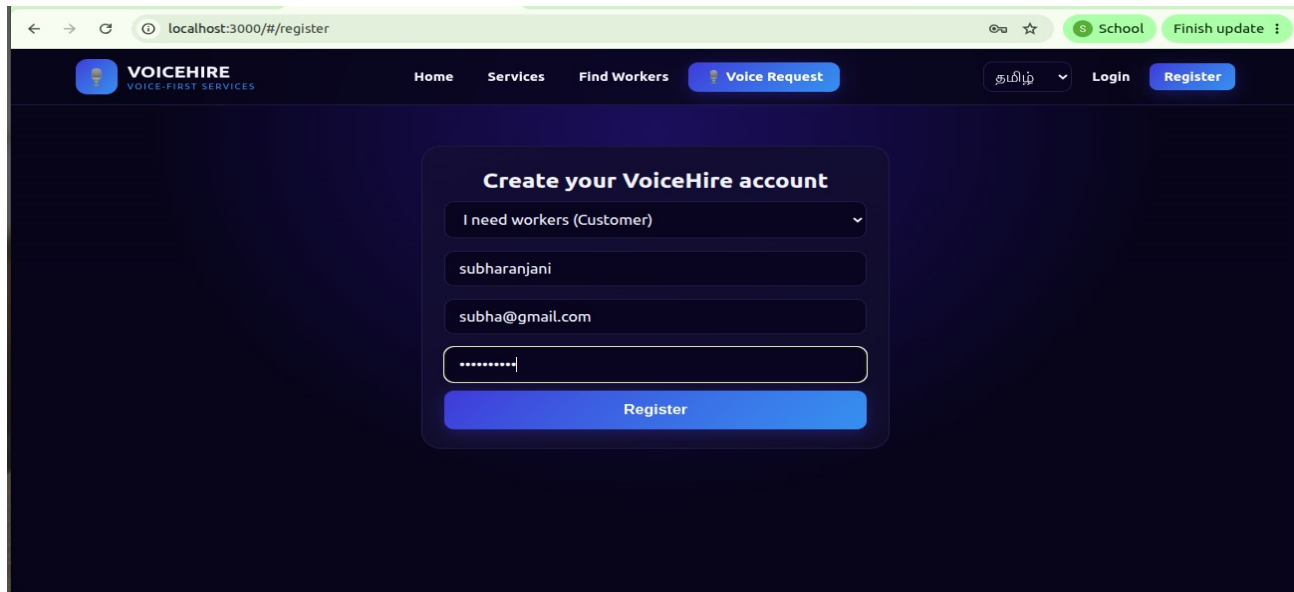
window.addEventListener('hashchange', route); route();
```

SCREENSHOTS:

1.HOME PAGE:



2.REGISTER PAGE(CUSTOMER):



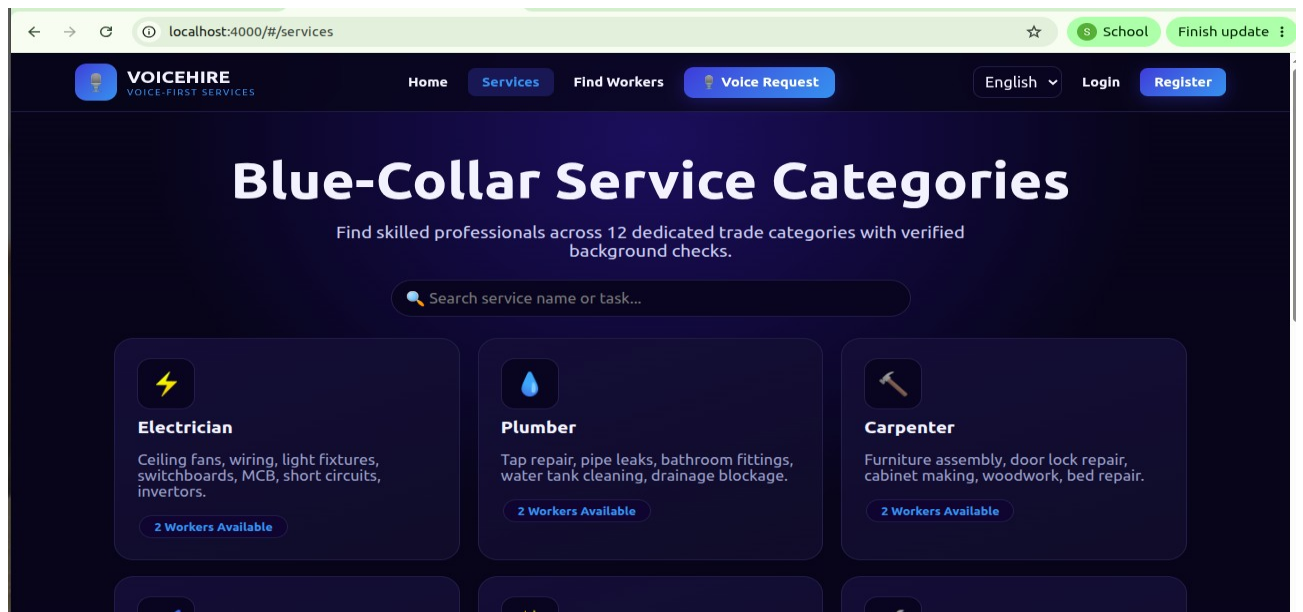
3.LOGIN PAGE(CUSTOMER):

The screenshot shows the login page for VoiceHire. The browser address bar indicates the URL is localhost:3000/#/login. The page has a dark blue header with the VoiceHire logo and navigation links: Home, Services, Find Workers, and Voice Request. There are also buttons for 'தமிழ்' (Tamil), 'Login', and 'Register'. The main content area is a dark blue card with a microphone icon and the title 'Login to VoiceHire'. Below the title, it says 'Select your account type or use 1-click academic demo logins below.' There are three tabs: 'Customer' (selected), 'Worker', and 'Admin'. Below the tabs, there are input fields for 'Email Address' (containing 'subha123@gmail.com') and 'Password' (masked with dots). A blue button labeled 'Login as customer' is below the password field. Below this, there is a lightning bolt icon and the text 'QUICK 1-CLICK ACADEMIC DEMO LOGIN'. At the bottom, there is a section for 'Customer Demo: Anand Sharma' with a 'Customer --' link.

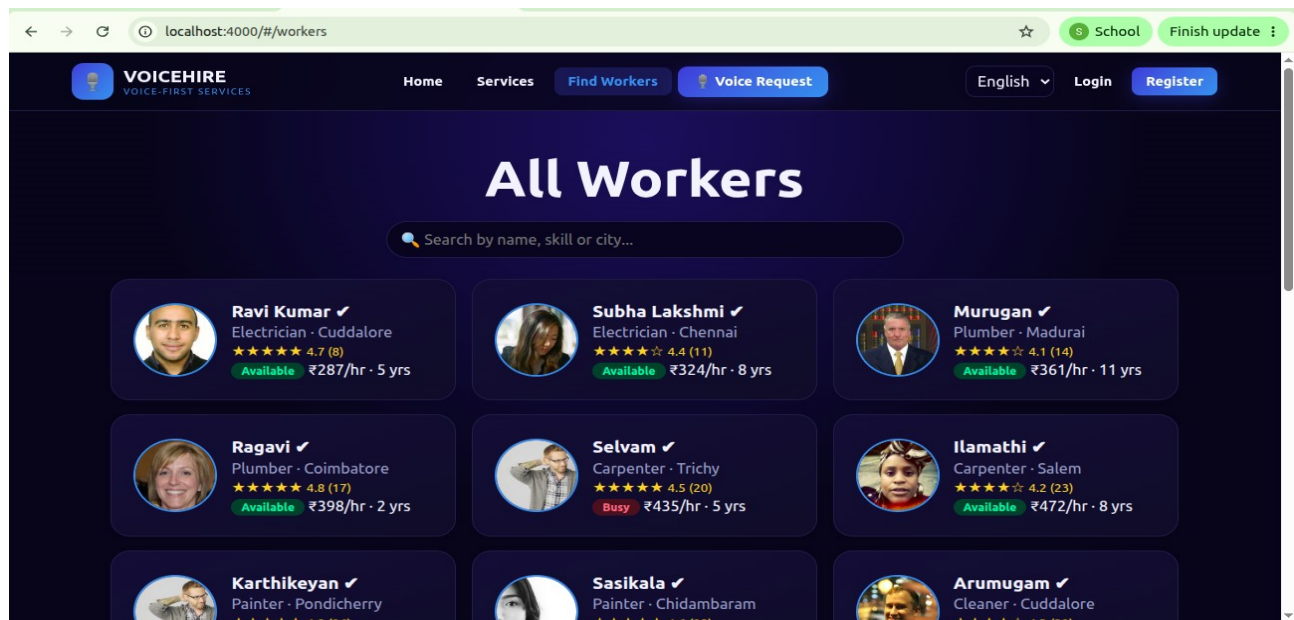
4.VOICE REQUEST PAGE:

The screenshot shows the voice request page for VoiceHire. The browser address bar indicates the URL is localhost:3000/#/voice. The page has a dark blue header with the VoiceHire logo and navigation links: Home, Services, Find Workers, and Voice Request. There are also buttons for 'தமிழ்' (Tamil), 'subharanjani', and 'Logout'. The main content area is a dark blue card with a lightning bolt icon and the text 'Multilingual Voice AI Recognizer'. Below this, it says 'Tell us what you need' and 'Click the microphone button and describe your service requirement naturally.' There is a microphone icon and the text 'Tap to speak (Tamil)'. Below this, there is a text input field with the example text 'e.g. My ceiling fan is not working / என் வீட்டில் குழாய் கசிகிறது'. A blue button labeled 'Find Workers' is below the input field.

5.SERVICES:



6.FINDING WORKING:



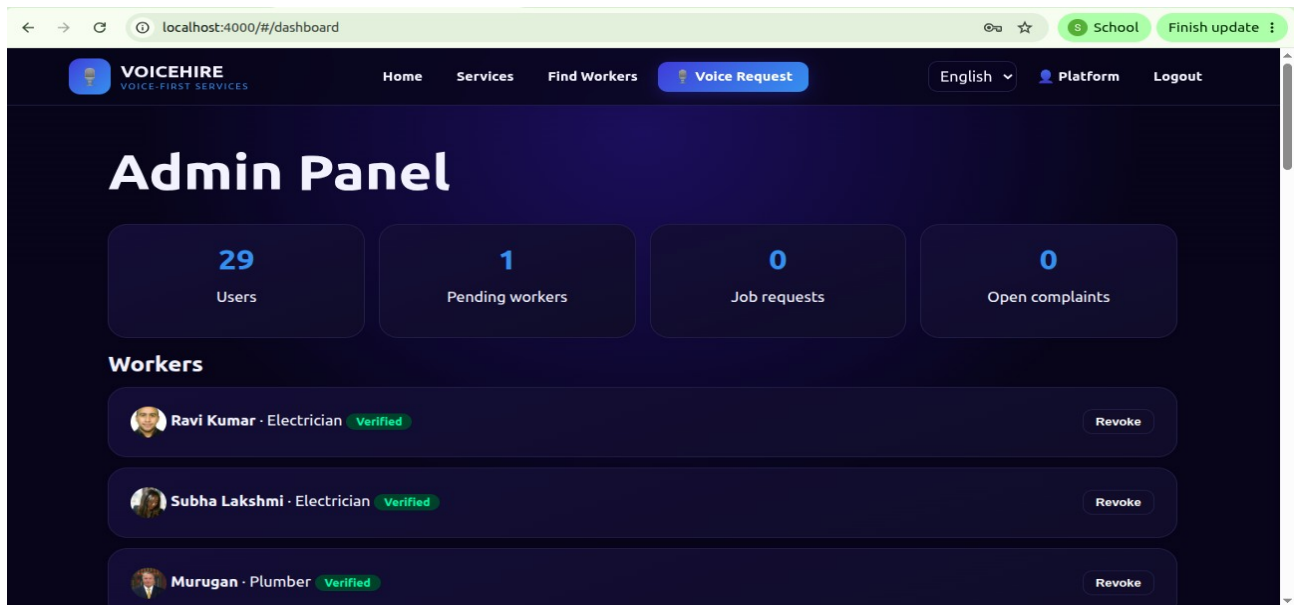
7.REGISTRATION (WORKERS):

The screenshot shows the 'Create your VoiceHire account' page for workers. The page has a dark blue header with the VoiceHire logo and navigation links: Home, Services, Find Workers, and Voice Request. There are also links for English, Login, and Register. The main content area is a form with the following fields: a dropdown menu for 'I am a worker (Job Seeker)', a text input for 'subha', a text input for 'subha567@gmail.com', a password input, a dropdown for 'Female', a dropdown for 'AC Technician', a text input for 'good,better,best skills', a text input for '5', a text input for 'erode', and a text input for '500l'. The form is set against a dark blue background.

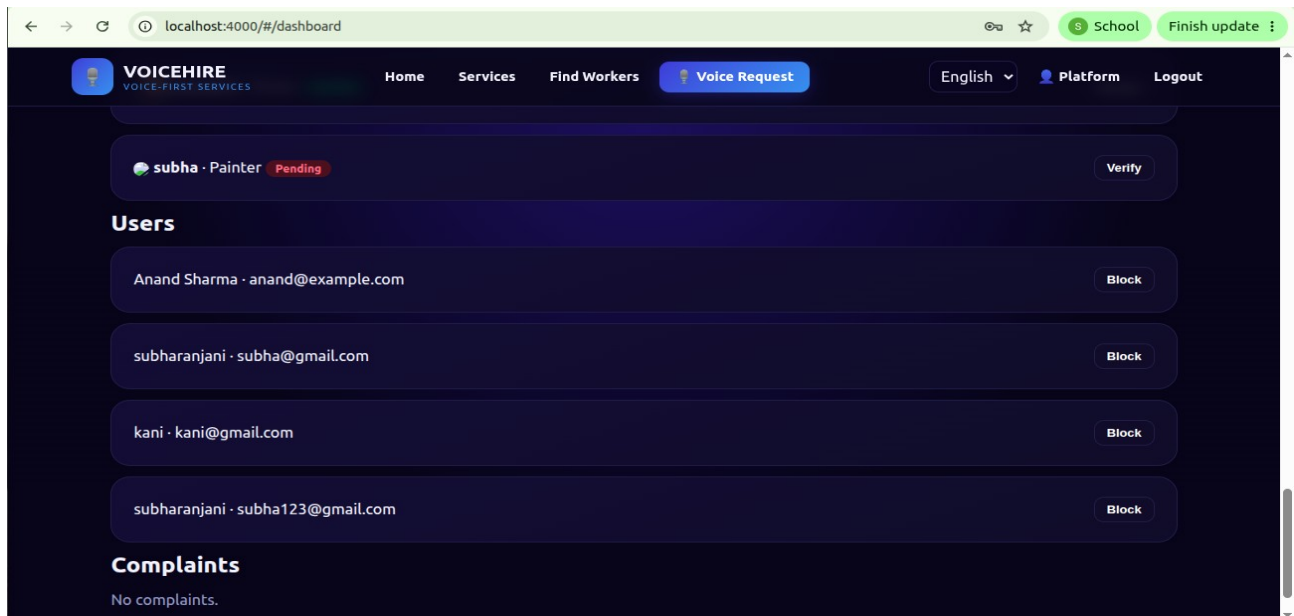
8.LOGIN PAGE(WORKER):

The screenshot shows the 'Login to VoiceHire' page. The page has a dark blue header with the VoiceHire logo and navigation links: Home, Services, Find Workers, and Voice Request. There are also links for English, Login, and Register. The main content area is a login form with the following elements: a dropdown menu for 'Customer', 'Worker', and 'Admin'; a text input for 'subha567@gmail.com'; a password input; a 'Login as worker' button; a 'QUICK 1-CLICK ACADEMIC DEMO LOGIN' link; and a 'Customer Demo: Anand Sharma' button. The form is set against a dark blue background.

9. ADMIN PANEL:



10. ADMIN PAGE:



E-RESOURCE:

GitHub link:

<https://github.com/Subharanjani-D/VoiceHire.git>